

***Question 1 : Comment définir l'instance du client `fetch` avec gestion des headers FHIR et du timeout ?**

***Réponse (Code) :**

```
````javascript
const BASE_URL = 'https://hapi.fhir.org/baseR4';

async function fhirFetch(endpoint, options = {}) {
 const controller = new AbortController();
 const timeoutId = setTimeout(() => controller.abort(), 8000);
 const response = await fetch(`${BASE_URL}${endpoint}`, {
 ...options,
 signal: controller.signal,
 headers: {
 'Accept': 'application/fhir+json',
 'Content-Type': 'application/fhir+json',
 ...options.headers
 }
 });
 clearTimeout(timeoutId);
 if (!response.ok) throw new Error(`Erreur FHIR: ${response.status}`);
 return response.json();
}
```

- Question 2 : Comment implémenter la fonction `searchPatients(query)` pour requêter `GET /Patient?name=` ?

- Réponse (Code) :

JavaScript

```
async function searchPatients(query) {
 if (!query) return [];
 const bundle = await
fhirFetch(`/Patient?name=${encodeURIComponent(query)}&_count=10`);
 return bundle.entry ? bundle.entry.map(e => e.resource) : [];
}
```

- Question 3 : Comment construire la fonction `displayName(patient)` pour extraire le nom et prénom ?

- Réponse (Code) :

JavaScript

```
function displayName(patient) {
 if (!patient?.name?.[0]) return 'Anonyme';
 const name = patient.name[0];
 const given = name.given ? name.given.join(' ') : '';
 const family = name.family || '';
 return `${given} ${family}`.trim();
}
```

```
}
```

- Question 4 : Comment créer la fonction `loadQuestionnaires(patientId)` pour récupérer les `QuestionnaireResponse` ?

- Réponse (Code) :

JavaScript

```
async function loadQuestionnaires(patientId) {
 const bundle = await
 fhirFetch(`/QuestionnaireResponse?patient=${patientId}&_sort=-authored`);
 return bundle.entry ? bundle.entry.map(e => e.resource) : [];
}
```

- Question 5 : Comment écrire l'algorithme `flattenItems(items)` pour extraire les questions/réponses imbriquées ?

- Réponse (Code) :

JavaScript

```
function flattenItems(items = []) {
 return items.reduce((acc, item) => {
 acc.push(item);
 if (item.item && item.item.length > 0) {
 acc.push(...flattenItems(item.item));
 }
 return acc;
 }, []);
}
```

- Question 6 : Comment implémenter la fonction `calculateHBI(qr)` calculant le score composite HBI ?

- Réponse (Code) :

JavaScript

```
function calculateHBI(qr) {
 const flat = flattenItems(qr.item || []);
 const getValue = (linkId) => {
 const found = flat.find(i => i.linkId === linkId);
 return found?.answer?.[0]?.valueInteger ?? 0;
 };
 const bienEtre = getValue('bien-etre');
 const douleur = getValue('douleur-abdominale');
 const selles = getValue('selles-liquides');
 const masse = getValue('masse-abdominale');
 const complications = getValue('complications');
```

```
return bienEtre + douleur + selles + masse + complications;
}
```

- Question 7 : Comment créer `analyseQuestionnaire(qr)` qui poste un `DiagnosticReport` et oriente le flux ?

- Réponse (Code) :

JavaScript

```
async function analyseQuestionnaire(qr) {
 const score = calculateHBI(qr);
 const report = {
 resourceType: 'DiagnosticReport',
 status: 'final',
 code: { text: 'Score HBI Maladie de Crohn' },
 subject: { reference: qr.subject.reference },
 conclusion: `Score HBI calculé : ${score}. ${score >= 5 ? 'Poussée sévère' : 'Remission'}`
 };

 const savedReport = await fhirFetch('/DiagnosticReport', {
 method: 'POST',
 body: JSON.stringify(report)
 });

 return { score, isSevere: score >= 5, reportId: savedReport.id };
}
```